PROMETEO

DISEÑO Y DESPLIEGUE DE UNA INFRAESTRUCTURA CLOUD (AWS) DE ALTA DISPONIBILIDAD PARA PLATAFORMAS E-LEARNING (MOODLE) EN AWS MEDIANTE INFRAESTRUCTURA COMO CÓDIGO (TERRAFORM)

Autor: David Arbelaez Mutis

Grado Superior en Administración de Sistemas Informáticos en Red (ASIR)

Prometeo (ThePower Education) - Campus Caja Mágica

Tutor: Maria Damia

Índice

01 Introducción

02 Objetivos

03 Análisis y planificación

04 Diseño de la solución

05 Desarrollo

06 Resultados

07 Conclusiones

08 Líneas futuras de mejora

09 Anexos

PROMETEO

1. Introducción

TFG

PROMETEO

01 INTRODUCCIÓN

1.1. Contexto del E-Learning

La disponibilidad de las plataformas de aprendizaje (LMS) se ha convertido en un factor crítico para las instituciones educativas. A diferencia de una web corporativa estática, un LMS como Moodle sufre picos de demanda extremos por ejemplo, durante un examen final que pueden tumbar servidores tradicionales. Las arquitecturas basadas en un único servidor (VPS) presentan un riesgo inasumible: si el hardware falla o el software se bloquea, el servicio se detiene por completo (Punto Único de Fallo o SPOF).

1.2. Antecedentes y Justificación (Evolución del Proyecto)

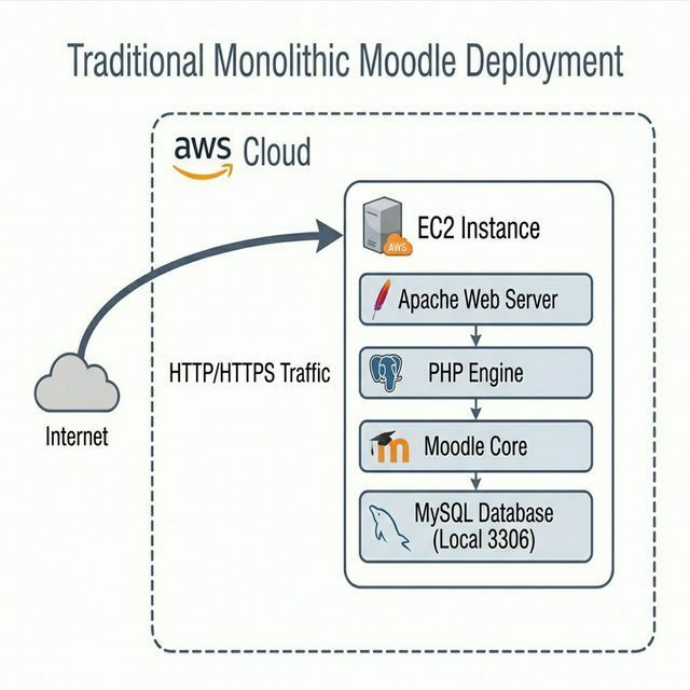
Este proyecto no es un ejercicio teórico, sino el resultado de un proceso de ingeniería iterativo realizado a lo largo de 2025.

Fase 1: El Prototipo Monolítico (Bitnami)

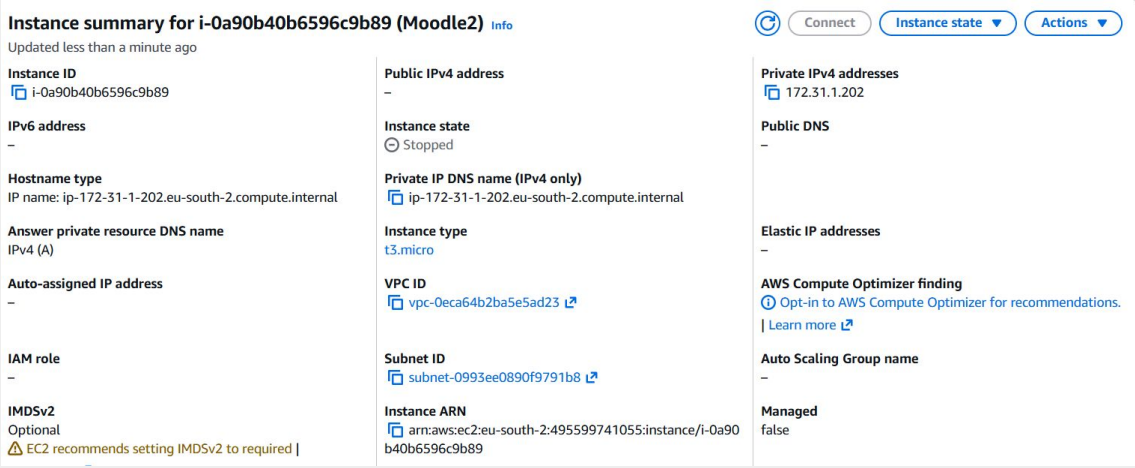
Inicialmente, se desplegó una solución basada en una imagen "Bitnami Moodle" en una única instancia EC2. Aunque funcional para pruebas de concepto, este modelo evidenció graves carencias:

1. **Rigidez:** La base de datos y la aplicación residían en el mismo servidor.
2. **Ineficiencia Económica:** Se detectó un sobrecoste mensual debido a la política de precios de AWS sobre las direcciones IPv4 públicas (Elastic IPs), necesarias para mantener el acceso al servidor único.
3. **Riesgo:** La actualización o fallo del servidor implicaba una parada total del servicio.

01 INTRODUCCIÓN



<**Imagen 1**. Arquitectura Monolítica Moodle>



<**Imagen 2.** Prototipo Monolítico Moodle>

PROMETEO

01 INTRODUCCIÓN

Detalles Técnicos del Hardware (Prototipo):

Para esta fase inicial se utilizó una instancia de la familia t3.micro con las siguientes especificaciones:

- **vCPU:** 2 vCPUs (Burstable Performance)
- **Memoria RAM:** 1 GiB
- **Almacenamiento:** Volumen EBS gp3 de 10 GB

Compute	Value
vCPUs	2
Memory (GiB)	1
Memory per vCPU (GiB)	0.5
Physical Processor	Intel Skylake E5 2686 v5
Clock Speed (GHz)	3.1 GHz
CPU Architecture	x86_64
GPU	0
GPU Average Wattage	0 W
GPU Architecture	none
Video Memory (GiB)	0
GPU Compute Capability (?)	0
FPGA	0
ffmpeg FPS	N/A
CoreMark iterations/Second	29069.767442

<**Imagen 3.** Detalles Técnicos del Hardware (Prototipo)>

PROMETEO

Fuente: https://instances.vantage.sh/aws/ec2/t3.micro?currency=USD

01 INTRODUCCIÓN

Este dimensionamiento se basó en pruebas iniciales de bajo tráfico. Sin embargo, el análisis de rendimiento bajo carga reveló limitaciones. Según la documentación oficial de Moodle y métricas de la comunidad, el consumo de memoria RAM por proceso de usuario concurrente oscila entre **50MB y 100MB**. Esto implica que una instancia de 1 GB de RAM (como la t3.micro), tras descontar la memoria del sistema operativo y la base de datos local, solo puede soportar de forma estable entre **10 y 20 usuarios concurrentes** simultáneos.

Esta limitación física del hardware fue un factor determinante para migrar hacia una arquitectura escalable horizontalmente (Auto-Scaling). Si bien el escalado vertical (aumentar RAM/CPU de una sola máquina) podría resolver temporalmente el problema de rendimiento, mantendría el problema crítico de disponibilidad, es decir, seguiría siendo un Punto Único de Fallo (SPOF). Por el contrario, el escalado horizontal mediante un Auto Scaling Group no solo permite sumar capacidad agregando nodos, sino que garantiza la redundancia y la tolerancia a fallos, distribuyendo la carga y asegurando que el servicio continúe incluso si un servidor individual falla.

01 INTRODUCCIÓN

Fase 2: La Solución Cloud Native (Terraform)

Basándose en el análisis anterior, se decidió reestructurar la solución hacia una arquitectura distribuida y automatizada. Se seleccionó **Terraform** para garantizar la replicabilidad del entorno y se migró a un modelo sin IPs públicas fijas en los servidores, delegando el acceso externo exclusivamente al Balanceador de Carga para mejorar la seguridad y eficiencia.

Es importante señalar que aunque el modelo *Pay-as-you-go* permite flexibilidad, la arquitectura distribuida final conlleva un coste base superior al modelo monolítico debido a la redundancia de componentes (RDS Multi-AZ, múltiples instancias EC2, EFS y ALB). Sin embargo, este incremento en el coste se justifica plenamente por la eliminación del Punto Único de Fallo y la capacidad de auto-recuperación, características indispensables para un entorno educativo crítico.

01 INTRODUCCIÓN

1.3. Alcance

Para garantizar la viabilidad y el rigor técnico del proyecto, es fundamental delimitar con precisión las fronteras de la implementación. El presente Trabajo de Fin de Grado se centra estrictamente en la ingeniería de infraestructura y operaciones (SysOps/Cloud), delimitándose de la siguiente manera:

1.3.1. Elementos incluidos en el alcance (In-Scope) El desarrollo técnico abarca el ciclo de vida completo de la infraestructura subyacente:

- **Aprovisionamiento de Red (Networking):** Diseño y despliegue de una Virtual Private Cloud (VPC) segmentada en subredes públicas y privadas distribuidas en múltiples Zonas de Disponibilidad (Multi-AZ).
- **Capa de Persistencia:** Despliegue de una base de datos relacional gestionada (Amazon RDS) y un sistema de archivos compartido elástico (Amazon EFS).
- **Capa de Computación y Balanceo:** Implementación de un Application Load Balancer (ALB) y un Grupo de Autoescalado (ASG) utilizando instancias efímeras.
- **Automatización Total (IaC):** Codificación de todos los recursos mencionados utilizando Terraform (HCL), junto con el scripting (*User Data*) necesario.
- **Pruebas de Resiliencia:** Ejecución de pruebas controladas de estrés y caída de nodos (*Chaos Testing*).
- **Análisis FinOps:** Auditoría de costes y toma de decisiones arquitectónicas basadas en el impacto financiero.

01 INTRODUCCIÓN

1.3.2. Elementos excluidos del alcance (Out of Scope) Las siguientes áreas quedan explícitamente fuera de la validación funcional de este TFG:

- **Seguridad de Capa 7 (HTTPS/SSL) y DNS:** La validación de la arquitectura se realiza mediante tráfico HTTP estándar a través del DNS público del ALB.
- **Administración Pedagógica de Moodle:** La plataforma se entregará con una instalación base y funcional (*vanilla*). No se realizará creación de cursos o integración con LDAP.
- **Pipelines CI/CD Completos:** La automatización se limitará al aprovisionamiento de infraestructura (Terraform CLI).
- **Ansible (Gestión de Configuración):** Aunque Terraform orquesta la infraestructura, la configuración del sistema operativo y los servicios se delega a scripts en Bash (User Data). La adopción de Ansible para la creación inmutable de AMIs y la ejecución de Playbooks queda fuera del alcance inicial de este prototipo.

2. Objetivos

TFG

PROMETEO

02 OBJETIVOS

2.1. Objetivo General

Diseñar, codificar e implementar una infraestructura de tres capas en AWS (Región España) capaz de alojar Moodle con tolerancia a fallos, escalado automático y persistencia de datos desacoplada, optimizando costes y rendimiento.

2.2. Objetivos Específicos

1. **Gestión de Infraestructura Local:** Habilitar y operar en la región eu-south-2 (España) para minimizar la latencia y cumplir con normativas de datos.

2. **Automatización (IaC):** Codificar el 100% de la infraestructura con Terraform, evitando configuraciones manuales.

3. **Alta Disponibilidad:** Implementar un Grupo de Autoescalado (ASG) que garantice un mínimo de dos servidores activos en diferentes zonas de disponibilidad.

4. **Persistencia:** Desacoplar la base de datos (RDS) y los archivos (EFS) de la capa de cómputo.

3. Análisis y planificación

TFG

PROMETEO

03 Análisis y planificación

3.1. Investigación Previa y Justificación Tecnológica

La definición de la arquitectura final no ha sido arbitraria, sino el resultado de un análisis comparativo basado tanto en la experiencia profesional previa del autor como en la investigación empírica realizada durante las fases iniciales de este proyecto.

3.1.1. Antecedentes Profesionales: Lecciones aprendidas en Entornos Azure

Como punto de partida para el diseño de la solución, se tomó como referencia la experiencia adquirida en despliegues anteriores de plataformas Moodle en entornos corporativos sobre Microsoft Azure. En dichos escenarios, se identificaron limitaciones críticas en arquitecturas monolíticas basadas en Windows Server:

1. **Sobrecarga del Sistema Operativo:** Se observó que el *overhead* (consumo de recursos base) de Windows Server requería instancias de mayor tamaño y coste para ofrecer el mismo rendimiento que una distribución Linux ligera.
2. **Complejidad de Gestión:** La administración de la pila PHP/Moodle sobre IIS (Internet Information Services) presentaba mayores fricciones de mantenimiento y automatización comparada con el estándar LAMP (Linux, Apache, MySQL, PHP) nativo.
3. **Escalabilidad Limitada:** Al mantener la base de datos y la aplicación en el mismo servidor, el escalado vertical (*resize*) implicaba ventanas de mantenimiento obligatorias y paradas de servicio, inaceptables para un entorno de alta disponibilidad.

03 Análisis y planificación

Decisión de Diseño: Basándose en esta experiencia previa, se estableció como requisito inicial para este TFG el uso de **Linux (Ubuntu Server)** y la migración a **AWS**, aprovechando su ecosistema maduro de automatización **(Terraform)** para resolver los problemas de escalabilidad detectados en el pasado.

3.1.2. Análisis del Prototipo Inicial en AWS (Fase Bitnami)

Ya en el marco de este proyecto, se realizó un despliegue piloto utilizando la imagen disponible en el marketplace de *Bitnami Moodle*. Aunque esta solución simplificó la instalación inicial, las pruebas de concepto revelaron nuevas ineficiencias que justificaron la evolución hacia la arquitectura final:

- **Ineficiencia Financiera (FinOps):** Se detectó que el modelo de servidor único exigía el uso de una **IP Elástica (IPv4 pública)** fija para garantizar la conectividad. Tras el cambio de políticas de precios de AWS, estas IPs generan un coste recurrente incluso si la instancia está detenida, lo que impacta negativamente en el presupuesto (**Imagen 4**).
- **Riesgo de Disponibilidad:** Se confirmó que la arquitectura "All-in-One" sigue siendo un Punto Único de Fallo (SPOF).

Esta validación práctica motivó el pivote hacia una arquitectura distribuida con **Balanceador de Carga (ALB)**, eliminando la necesidad de IPs públicas en los servidores y delegando la entrada de tráfico a un componente de alta disponibilidad.

03 Análisis y planificación

<**Imagen 4**. Costos Moodle Bitnami Uso IP Publica>

PROMETEO

03 Análisis y planificación

3.1.3. Requisitos de Hardware y Dimensionamiento (Sizing)

Para determinar el tamaño de instancia adecuado (t3.micro), se contrastó el rendimiento del prototipo con la documentación oficial de Moodle.

Según las guías de rendimiento (*Performance Recommendations*), el consumo de memoria RAM es el factor limitante principal, estimándose un uso de **50 MB a 100 MB por proceso de usuario concurrente**.

- **Escenario Monolítico:** En una instancia con 1 GB de RAM, el sistema operativo y el motor de base de datos local consumen más del 60% de la memoria disponible, dejando margen para apenas 3-5 usuarios concurrentes antes de usar memoria de intercambio (swap) y degradar el servicio.
- **Escenario Desacoplado (Arquitectura Propuesta):** Al extraer la base de datos a **Amazon RDS**, se libera la memoria de la instancia EC2 para dedicarla exclusivamente a procesos PHP de Moodle, multiplicando la capacidad de concurrencia del mismo hardware.

03 Análisis y planificación

3. 2 Requisitos de Dimensionamiento para Moodle

El cálculo de recursos para Moodle no es lineal y depende fuertemente de la concurrencia. Según las recomendaciones de rendimiento de Moodle (*Performance Recommendations*), una regla general aceptada es asignar aproximadamente **1 GB de RAM por cada 10 a 20 usuarios concurrentes** activos (no registrados).

En un entorno monolítico, el servidor debe gestionar tanto la carga web (Apache/PHP) como la base de datos (MySQL), lo que aumenta drásticamente el consumo de memoria. Al desacoplar la base de datos a un servicio externo (RDS), se libera memoria en los servidores web, permitiendo que instancias pequeñas como las t3.micro puedan manejar más tráfico de usuarios antes de requerir escalado.

What hardware should I buy for 'n' concurrent users?

Firstly see above to make sure you express 'n' somewhere near correctly. Unfortunately, the answer is "it depends". It will depend primarily on your chosen operating system, your software configuration (especially web server and database tuning) and what the users will be doing. The worst possible scenario is a large class all starting a timed quiz at exactly the same time. Luckily, users sat reading stuff are not generating any server load at all (until they click for the next activity/page). Very roughly, worst case, your Moodle site may only handle as few as 10-20 concurrent users per GB of memory. Moodle is a large application and can easily use more that 50MB (sometimes a lot more) of RAM per process. If you are planning a very large site, you are strongly encouraged to run a pilot first and take some measurements or resources used.

Prospective Moodle users tend to vastly over-estimate the required concurrency. It's potentially a very expensive mistake if you get this wrong.

<Imagen 3. Documentación Moodle usuarios concurrentes 10-20 por GB de memoria RAM>

PROMETEO

Fuente: https://docs.moodle.org/501/en/Performance_FAQ

03 Análisis y planificación

3.3 Tecnologías seleccionadas y justificación

La consecución de los objetivos de este proyecto requiere el uso de herramientas robustas, que representan el estándar actual de la industria tecnológica. La selección de cada componente no ha sido arbitraria, sino que responde a la necesidad de construir un entorno escalable, seguro y automatizado. A continuación, se detalla el ecosistema tecnológico empleado:

3.3.1. Proveedor de Infraestructura: Amazon Web Services (AWS)

AWS se posiciona como el proveedor líder de nube pública a nivel global. Para este proyecto, la elección de AWS frente a otras alternativas (como Azure o Google Cloud) se fundamenta en dos grandes pilares:

- **Madurez en Servicios Gestionados (PaaS):** En lugar de limitarnos a desplegar máquinas virtuales puras (IaaS), AWS permite delegar la administración operativa de componentes críticos. Por ejemplo, el uso de **Amazon RDS (Relational Database Service)** elimina la necesidad de parchear, asegurar y hacer backups manuales del motor de base de datos, tareas que consumen mucho tiempo en entornos tradicionales.
- **Presencia Local (Región eu-south-2):** La reciente apertura de la región de AWS en España (Aragón) permite alojar la infraestructura físicamente en territorio nacional. Esto es fundamental para un entorno educativo, ya que reduce drásticamente la latencia de red para los alumnos locales y garantiza el cumplimiento normativo estricto del RGPD (Reglamento General de Protección de Datos) europeo.

03 Análisis y planificación

3.3.2. Orquestación y Automatización: HashiCorp Terraform

Para abandonar la práctica ineficiente de configurar la infraestructura manualmente a través de la consola web (práctica conocida peyorativamente como *"ClickOps"*), se ha adoptado el paradigma de Infraestructura como Código (IaC).

- **¿Qué es y por qué se eligió?** Terraform es una herramienta de código abierto que permite definir recursos en la nube mediante archivos de texto utilizando el lenguaje declarativo HCL (*HashiCorp Configuration Language*). Se eligió por encima de soluciones nativas como AWS CloudFormation debido a su enfoque agnóstico: el conocimiento adquirido con Terraform es transferible a otros proveedores cloud en el futuro.

- **Papel en el proyecto:** Terraform actúa como el "director de orquesta". Su función es leer los archivos de configuración y comunicarse con la API de AWS para crear la red, levantar los servidores, configurar los firewalls y montar los discos, garantizando que el entorno sea 100% reproducible y auditable mediante control de versiones (Git).

03 Análisis y planificación

3.3.3 Aplicación Core: Moodle (Stack LAMP)

Moodle es el Sistema de Gestión de Aprendizaje (LMS) de código abierto más utilizado a nivel mundial en el ámbito educativo. Su arquitectura de software dicta directamente las necesidades de la infraestructura subyacente que hemos diseñado:

- **El Stack Tecnológico:** Moodle requiere un entorno LAMP (Linux, Apache, MySQL, PHP). Para funcionar correctamente en un clúster de alta disponibilidad, Moodle exige que todos los servidores web compartan exactamente la misma base de datos y un mismo directorio de archivos físicos llamado **moodledata** (donde se guardan los PDFs, vídeos y entregas de los alumnos).
- **Justificación arquitectónica:** Es precisamente esta exigencia de Moodle la que justifica la inclusión de un servicio de base de datos externa (RDS) y un sistema de archivos en red NFS (Amazon EFS) en nuestro diseño, permitiendo que múltiples servidores procesen peticiones simultáneamente sin corromper la información de los cursos.

03 Análisis y planificación

3.3.4. Patrón de Diseño: Arquitectura Multi-AZ (Multi-Zona de Disponibilidad)

Más que una herramienta de software, esta es la decisión de diseño de red más importante del proyecto.

- **¿En qué consiste?** Una región de AWS está compuesta por múltiples "Zonas de Disponibilidad" (AZ). Cada AZ es uno o varios centros de datos físicos distintos, separados por kilómetros de distancia, con fuentes de alimentación, refrigeración y conectividad de red totalmente independientes.

- **El valor aportado:** Al diseñar la Virtual Private Cloud (VPC) distribuyendo los recursos equitativamente entre la Zona A y la Zona B, logramos tolerancia a desastres físicos. Si un centro de datos entero de AWS sufre un apagón o un incendio, el Balanceador de Carga (ALB) enrutará automáticamente a los alumnos hacia los servidores de la zona superviviente. Esto transforma un sistema vulnerable en un servicio resiliente de grado empresarial, eliminando el Punto Único de Fallo (SPOF).

03 Análisis y planificación

Enfoque Metodológico (DevOps & FinOps)

Para llevar a cabo este proyecto, se ha descartado el uso de metodologías tradicionales "en cascada" (donde las fases avanzan de manera rígida y no se puede retroceder si hay errores) en favor de un enfoque ágil y progresivo. Esto significa que la infraestructura se ha construido paso a paso, realizando pequeñas pruebas continuas para adaptar el diseño y mejorarlo sobre la marcha según los resultados obtenidos.

Desde el primer momento, el trabajo se ha guiado por dos grandes pilares: por un lado, la mentalidad DevOps, que nos lleva a tratar toda la infraestructura tecnológica como si fuera código de programación (evitando las configuraciones manuales propensas a errores) y por otro lado, la cultura FinOps, analizando el impacto económico de cada recurso en la nube antes de desplegarlo. Asimismo, el diseño se apoya firmemente en la filosofía "Cloud-Native First", lo que significa que se ha dado prioridad absoluta al uso de servicios gestionados de AWS para reducir drásticamente la carga de mantenimiento manual.

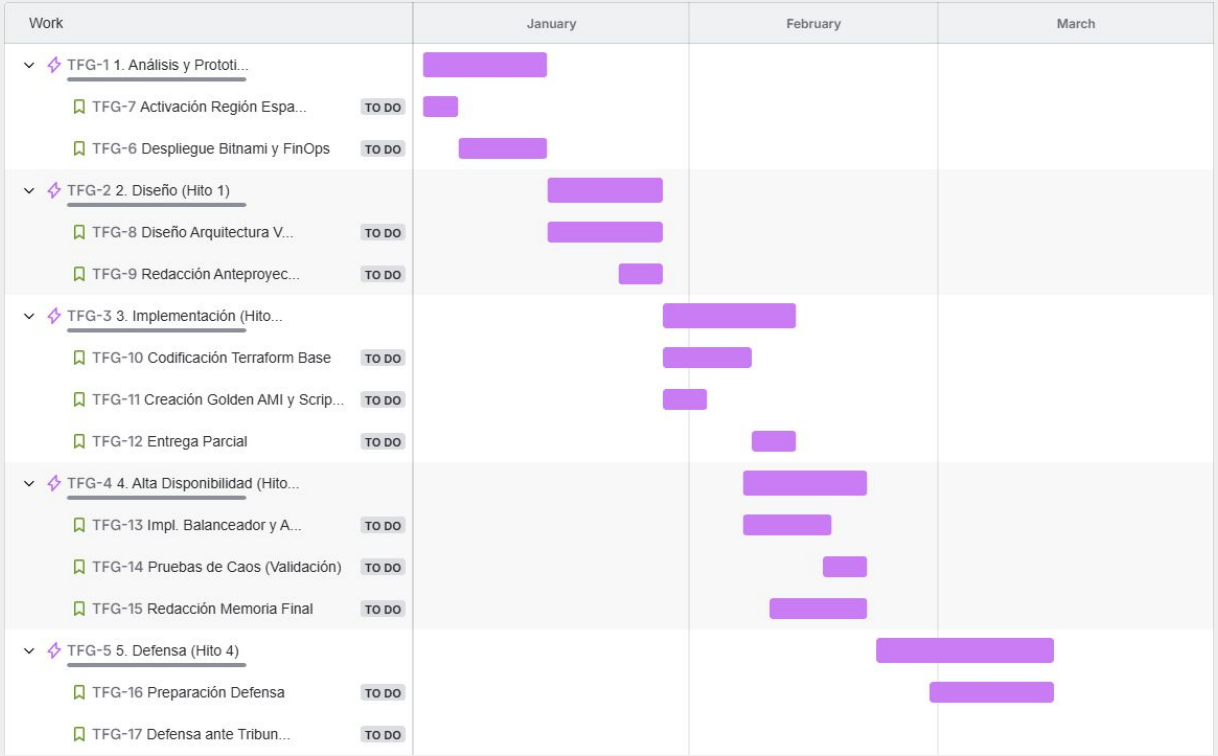
03 Análisis y planificación

Enfoque Metodológico (DevOps & FinOps)

Las fases metodológicas se han estructurado en sprints técnicos:

1. **Fase de Descubrimiento y Viabilidad (FinOps):** Validación económica y técnica de la región eu-south-2 y detección de ineficiencias en el prototipo inicial.
2. **Fase de Diseño Arquitectónico (Well-Architected):** Diseño basado en los pilares de Fiabilidad (Multi-AZ) y Seguridad (Mínimo Privilegio).
3. **Fase de Implementación (IaC):** Desarrollo modular con Terraform, codificando componentes de red, datos y computación de forma aislada.
4. **Fase de Validación y Resiliencia (Chaos Engineering):** Inyección controlada de fallos para medir la capacidad de recuperación del sistema.

03 Análisis y planificación



<Imagen 4. Diagrama Gantt>

PROMETEO

03 Análisis y planificación

Key	Parent	Summary	Start date	Due date
TFG-1		1. Análisis y Prototipo	02/01/2026	15/01/2026
TFG-7	TFG-1	Activación Región España	02/01/2026	05/01/2026
TFG-6	TFG-1	Despliegue Bitnami y FinOps	06/01/2026	15/01/2026
TFG-2		2. Diseño (Hito 1)	16/01/2026	28/01/2026
TFG-8	TFG-2	Diseño Arquitectura VPC	16/01/2026	28/01/2026
TFG-9	TFG-2	Redacción Anteproyecto	24/01/2026	28/01/2026
TFG-3		3. Implementación (Hito 2)	29/01/2026	12/02/2026
TFG-10	TFG-3	Codificación Terraform Base	29/01/2026	07/02/2026
TFG-11	TFG-3	Creación Golden AMI y Scripts	29/01/2026	02/02/2026
TFG-12	TFG-3	Entrega Parcial	08/02/2026	12/02/2026
TFG-4		4. Alta Disponibilidad (Hito 3)	07/02/2026	20/02/2026
TFG-13	TFG-4	Impl. Balanceador y ASG	07/02/2026	16/02/2026
TFG-14	TFG-4	Pruebas de Caos (Validación)	16/02/2026	20/02/2026
TFG-15	TFG-4	Redacción Memoria Final	10/02/2026	20/02/2026
TFG-5		5. Defensa (Hito 4)	22/02/2026	01/04/2026
TFG-16	TFG-5	Preparación Defensa	28/02/2026	13/03/2026
TFG-17	TFG-5	Defensa ante Tribunal	30/03/2026	30/03/2026

<Imagen 5. Fases e hitos>

4. Diseño de la solución

TFG

PROMETEO

04 Diseño de la solución

4.1. DISEÑO DEL SISTEMA Y ARQUITECTURA

Al tratarse de un proyecto de ingeniería de infraestructura (Cloud y SysOps), el diseño del sistema no se centra en la programación visual de una aplicación desde cero, sino en la orquestación, interconexión, resiliencia y despliegue de los servicios subyacentes que dan soporte a la plataforma educativa. Bajo esta premisa, la arquitectura ha sido concebida alineándose estrictamente con los principios del Well-Architected Framework. Esto nos permite garantizar un entorno robusto y escalable, fundamentando el diseño de la plataforma en los siguientes pilares clave:

- **Fiabilidad:** Arquitectura Multi-AZ y Auto-Healing.
- **Optimización de Costes:** Prácticas FinOps y Auto-Scaling.
- **Excelencia Operativa:** Infraestructura como Código (Terraform).

A continuación, se detallan los pilares de este diseño adaptados a los estándares de la nube.

04 Diseño de la solución

4.2. Arquitectura General

Este proyecto no es un ejercicio teórico, sino el resultado de un proceso realizado a lo largo de 2025 y 2026. El diseño general rompe con el modelo tradicional monolítico (donde todo reside en un solo servidor) y evoluciona hacia una arquitectura nativa en la nube de tres capas (3-Tier), altamente disponible y desacoplada:

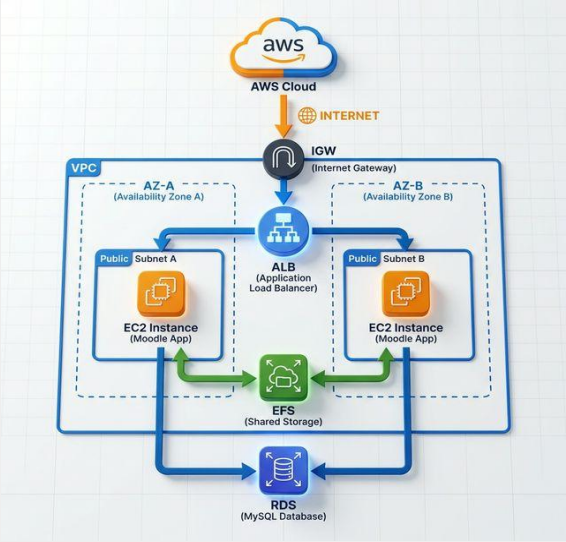
- **Capa de Distribución (Pública):** Formada por el Application Load Balancer (ALB). Es la única parte expuesta a Internet y se encarga de recibir el tráfico de los alumnos y repartirlo de forma equilibrada.
- **Capa de Computación (Privada):** Compuesta por el Auto Scaling Group (ASG) con instancias EC2 efímeras. Aquí es donde se ejecuta el código PHP de Moodle y el servidor web Apache. Si hay mucha demanda, esta capa crece dinámicamente; si no, se encoge para optimizar el gasto (FinOps).
- **Capa de Persistencia (Privada):** Donde residen los datos de forma centralizada y segura. Se divide en Amazon RDS (para la base de datos MySQL) y Amazon EFS (almacenamiento de red NFS compartido para guardar los PDFs, imágenes y entregas de los cursos).

Esta separación garantiza que, si la capa de computación sufre una caída masiva, el núcleo de los datos educativos permanezca intacto y el servicio pueda restablecerse automáticamente.

04 Diseño de la solución

4.3. Diagramas y Flujos del Sistema

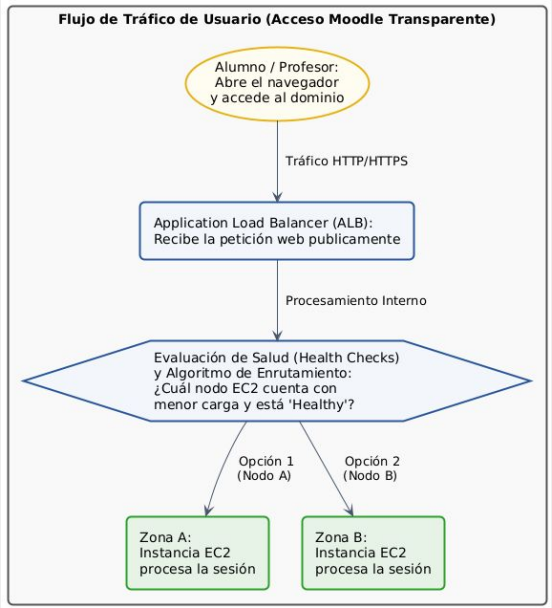
- **Diagrama de Componentes (Topología de Red):** Define las fronteras de red mediante la Virtual Private Cloud (VPC). Se ha diseñado un aislamiento estricto donde las instancias web y las bases de datos operan en subredes privadas. La comunicación se regula mediante Grupos de Seguridad (Security Groups) que actúan como firewalls de estado: la base de datos solo "escucha" a los servidores web, y estos solo admiten peticiones validadas por el Balanceador.



<**Imagen 6.** Diagrama de Componentes (Topología de Red)>

04 Diseño de la solución

- **Flujo de Tráfico de Usuario (Acceso Moodle):** El diseño contempla el viaje de la petición HTTP. El usuario accede al dominio público del ALB, este evalúa los Health Checks de los nodos en la Zona A y la Zona B, enrutando la sesión al servidor EC2 con menor carga. Para el alumno o profesor, la complejidad de esta distribución multinodo es completamente transparente.



<**Imagen 7.** Flujo de Peticiones HTTP>

PROMETEO

04 Diseño de la solución

- **Flujo de Autorreparación (Auto-Healing):** Este es el diagrama de comportamiento interno más crítico. Se ha diseñado un flujo donde, si un nodo EC2 deja de responder (simulando un fallo de hardware), el ALB corta el tráfico hacia él, el ASG detecta la anomalía, destruye la instancia corrupta y lanza una nueva basada en la Golden AMI. Al arrancar, el User Data entra en acción montando el disco EFS, reincorporando el nodo al clúster en minutos sin intervención humana.

<**Imagen 8.** Flujo de Autorreparación (Auto-Healing)>

04 Diseño de la solución

4.4. Diseño de Interfaz Infraestructura Cloud

En un proyecto de arquitectura en la nube, el concepto de "interfaz" trasciende la pantalla del usuario final para abarcar las herramientas de gestión, control y operación del sistema. El diseño de interfaces se ha estructurado en tres niveles:

- **Interfaz de Usuario Final (Frontend LMS):** Se ha adoptado y desplegado la interfaz nativa de Moodle. Este diseño garantiza un entorno responsive (adaptable a dispositivos móviles, tablets y escritorio) y ofrece una experiencia de usuario (UX) madura y estandarizada en el sector educativo, reduciendo a cero la curva de aprendizaje para la comunidad estudiantil.
- **Interfaz de Operaciones (Backend IaC):** La interfaz principal para el administrador del sistema no es una consola gráfica, sino el código declarativo (Terraform/HCL). Se ha diseñado una estructura de archivos (main.tf, variables.tf, security.tf) modular y semánticamente limpia. Esta "interfaz de código" permite leer, auditar y modificar la arquitectura entera de forma predecible, controlada por versiones (Git).
- **Interfaz de Monitorización y Control (Dashboards):** Se ha integrado el uso de la consola de AWS, específicamente Amazon CloudWatch y AWS Cost Explorer, como interfaces visuales para la toma de decisiones. Estos paneles permiten visualizar en tiempo real las métricas de saturación de CPU, los picos de tráfico de red y el gasto operativo (FinOps), actuando como el panel de instrumentos principal del sistema.

04 Diseño de la solución

4.5. Consideraciones de Accesibilidad y Usabilidad

El diseño de una infraestructura robusta impacta de manera directa y fundamental en cómo los usuarios finales y los administradores interactúan con la plataforma. Para este TFG, se han aplicado los siguientes criterios de usabilidad y accesibilidad:

- **Disponibilidad (Accesibilidad al servicio):** La principal barrera de accesibilidad en el E-Learning ocurre cuando "el campus virtual se cae" por exceso de tráfico (ej. fechas límite de entregas). El diseño Multi-AZ y el Autoescalado resuelven este problema de raíz, asegurando que el acceso a la educación esté garantizado 24/7 sin importar la carga concurrente.

- **Rendimiento y Baja Latencia (Usabilidad):** Al haber gestionado la activación e implementación en la región eu-south-2 (España - Aragón), se ha optimizado el recorrido de los paquetes de red. Esto se traduce en una carga de páginas y subida de archivos (tareas, vídeos) casi instantánea, mejorando drásticamente la experiencia del usuario local frente a alojamientos internacionales.

- **Accesibilidad Inclusiva (WCAG 2.1)**: Al utilizar el core oficial de Moodle como interfaz pública, el proyecto hereda su cumplimiento nativo con las pautas de accesibilidad Web (WCAG). Esto garantiza que el sistema sea navegable por personas con discapacidades visuales o motoras, soportando lectores de pantalla, alto contraste y navegación integral por teclado.

04 Diseño de la solución

- **Usabilidad Operativa (Developer Experience - DX):** La adopción de Terraform proporciona una alta "usabilidad" para el equipo de sistemas. Al evitar la configuración manual ("ClickOps"), cualquier nuevo administrador puede entender la topología exacta del proyecto simplemente leyendo los archivos .tf. Se han incluido descripciones claras, variables comentadas y un script de automatización (user_data) que elimina el error humano, facilitando enormemente el mantenimiento del ciclo de vida del proyecto.

5. Desarrollo

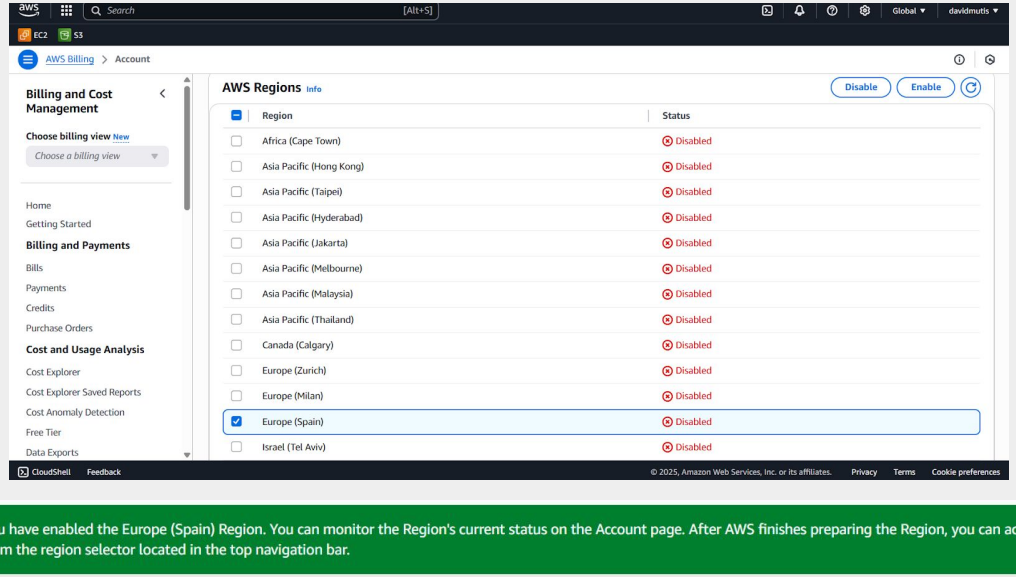
TFG

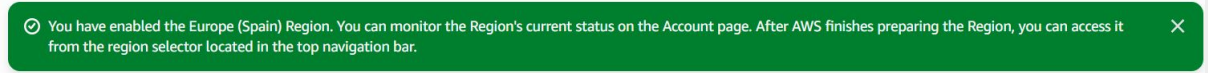
PROMETEO

05 DESARROLLO E IMPLEMENTACIÓN TÉCNICA

5.1. Gestión del Tenant: Activación de Región Local

Un obstáculo inicial fue el estado de la región Europe (Spain) eu-south-2. Al tratarse de una región Opt-in (de activación voluntaria), los recursos no podían desplegarse por defecto. Fue necesario realizar un procedimiento administrativo de activación y esperar a la propagación de los cambios en la cuenta de AWS. La elección de la región local responde a criterios de soberanía de datos y reducción de latencia para el usuario final.





<**Imagen 9:** Panel de AWS mostrando el cambio de estado de la región de "Disabled" a "Enabled".

PROMETEO

05 DESARROLLO E IMPLEMENTACIÓN TÉCNICA

5.1.2 Gestión del Tenant: Análisis FinOps - El impacto de las IPs Públicas

Durante la fase de prototipado, la monitorización en AWS Cost Explorer reveló un patrón de gasto ineficiente. Una parte significativa del coste mensual provenía de la partida EC2-Other, específicamente atribuible a las direcciones IPv4 públicas reservadas para el servidor monolítico.

Al identificar esta ineficiencia, se tomó la decisión de liberar la IP pública a principios de diciembre. Como se evidencia en la gráfica de costes, esta acción redujo el gasto operativo, validando la necesidad de una arquitectura donde los servidores de backend no posean IPs públicas fijas, sino que sean accedidos a través de un recurso compartido (el Balanceador).

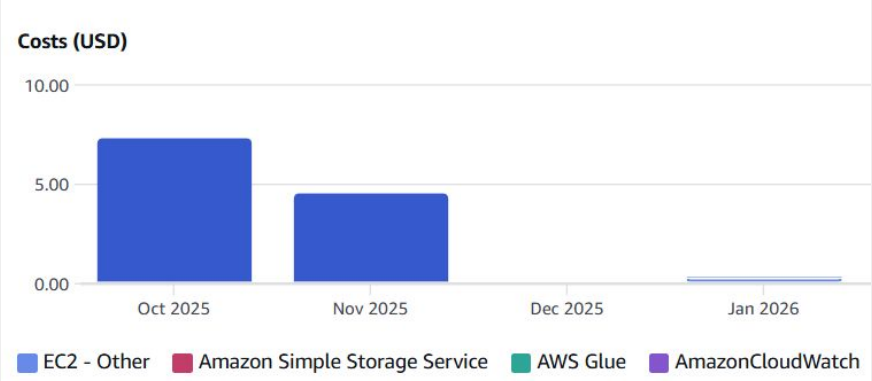


Imagen 10: Evolución de costes mensuales. Se aprecia la caída del gasto en diciembre tras la optimización de recursos de red.

05 DESARROLLO E IMPLEMENTACIÓN TÉCNICA

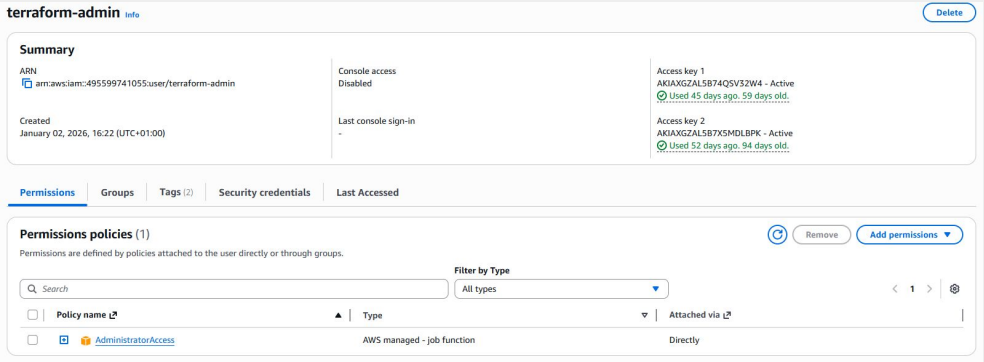
5.2. Gobernanza de Despliegue

Para iniciar el despliegue de la infraestructura, el primer paso fundamental fue el acondicionamiento de la estación de trabajo, instalando y validando las versiones de AWS CLI v2.32.26 y Terraform v1.14.8 Siguiendo las directrices del 'Well-Architected Framework' de AWS, se evitó por completo el uso de la cuenta raíz (Root Account) para las operaciones de desarrollo, en su lugar, se procedió a la creación de un usuario específico dentro de Identity and Access Management (IAM). A este usuario se le otorgaron permisos administrativos limitados y se le generaron credenciales de acceso (Access Keys) para permitir la gestión automatizada de recursos desde la terminal. La vinculación entre el entorno local y el Tenant de AWS se materializó mediante el comando **aws configure**, estableciendo una comunicación cifrada y trazable que permitió a Terraform orquestar los recursos en la región de España (eu-south-2) de forma segura y profesional.

```
C:\Users\David>aws --version
aws-cli/2.32.26 Python/3.13.11 Windows/11 exe/AMD64

C:\Users\David>terraform -version
Terraform v1.14.8
on windows_amd64
```

<**Imagen 10:** Versiones de Terraform y AWS Cli



<**Imagen 11:** Usuario terraform-admin

PROMETEO

05 DESARROLLO E IMPLEMENTACIÓN TÉCNICA

5.3 Infraestructura como Código (IaC): Arquitectura en Terraform

La implementación de la infraestructura no se realizó de forma manual a través de la consola web, sino mediante Infraestructura como Código (IaC) utilizando Terraform. Este enfoque permite que toda la arquitectura (red, servidores, bases de datos y seguridad) esté definida en archivos de texto, lo que garantiza la repetibilidad, evita errores humanos y permite mantener un control de versiones exacto de cada cambio realizado.

La arquitectura del proyecto se ha modularizado en los siguientes archivos de configuración, cumpliendo con una estructura organizada y profesional:

- **provider.tf:** Establece la conexión con la plataforma de AWS y fija la región de despliegue en España (eu-south-2). Esto asegura que todos los recursos se creen en el mismo centro de datos local.

- **variables.tf** y **outputs.tf:** Actúan como el panel de control del proyecto. El primero permite parametrizar datos sensibles y configuraciones (como tipos de instancia o contraseñas) sin modificar el código principal. El segundo devuelve información crítica al finalizar el proceso, como la URL pública del balanceador para acceder a Moodle.

- **main.tf:** Es el archivo encargado de la red troncal. Define la VPC (Virtual Private Cloud) y segmenta el espacio de direcciones en subredes públicas y privadas, orquestando la base donde residirán todos los servicios.

05 DESARROLLO E IMPLEMENTACIÓN TÉCNICA

5.3 Infraestructura como Código (IaC): Arquitectura en Terraform

- **security.tf:** Centraliza la seguridad lógica mediante la definición de Security Groups. Aquí se programan las reglas de firewall que permiten, por ejemplo, que la base de datos solo acepte conexiones desde los servidores de Moodle y no desde internet.

- **storage.tf:** Gestiona la capa de datos. Define la base de datos Amazon RDS (MySQL) y el sistema de archivos Amazon EFS. Este archivo asegura que el almacenamiento sea persistente y esté disponible para todos los nodos.

- **compute.tf:** Define las características de los servidores, incluyendo la elección de la imagen (AMI) y el tipo de instancia, asegurando que el hardware virtual sea el adecuado para el rendimiento esperado.

- **alb.tf** y **asg.tf:** Configuran la inteligencia del sistema. El alb.tf gestiona el Application Load Balancer para repartir el tráfico, mientras que el asg.tf define el Auto Scaling Group, estableciendo las reglas para que el sistema cree o destruya servidores automáticamente según la demanda de los alumnos.

5.3.1 Infraestructura como Código (IaC): El Ciclo de Vida del Despliegue

Para materializar esta arquitectura, se siguió el flujo de trabajo estándar de Terraform:

- **terraform init:** Para inicializar el directorio y descargar los conectores necesarios.
- **terraform plan:** Para previsualizar los cambios y asegurar que la infraestructura real coincidirá con lo diseñado antes de ejecutar cualquier acción.
- **terraform apply:** Para realizar el despliegue automatizado en la nube.
- **terraform destroy:** Comando fundamental para la finalización del proyecto o limpieza del entorno. Permite eliminar de forma automatizada y ordenada la totalidad de los recursos creados, asegurando que no queden servicios activos que generen costes innecesarios tras concluir las pruebas o la defensa del proyecto.

Este método de trabajo genera un archivo de estado llamado **terraform.tfstate**, que actúa como la "única fuente de verdad", permitiendo que el sistema sepa en todo momento qué recursos existen y cómo deben actualizarse de forma coherente.

05 DESARROLLO E IMPLEMENTACIÓN TÉCNICA

5.4. Diseño de la Arquitectura de Red (VPC)

Se definió una Virtual Private Cloud (VPC) con un espacio de direccionamiento 10.0.0.0/16. Se optó por una máscara amplia (/16) siguiendo las buenas prácticas de diseño de AWS, lo que garantiza escalabilidad futura para añadir nuevas subredes sin necesidad de rediseñar la red base.

La topología se segmentó en cuatro subredes distribuidas en dos Zonas de Disponibilidad (eu-south-2a y eu-south-2b) para cumplir con el requisito de alta disponibilidad:

- **Capa Pública:** Subredes para el Balanceador de Carga.
- **Capa Privada:** Subredes aisladas para la base de datos y almacenamiento.

5.5. Capa de Persistencia de Datos

Para solucionar el problema de volatilidad de los datos en servidores efímeros, se implementaron dos servicios clave:

- **Base de Datos:** Se desplegó Amazon RDS (MySQL 8.0) en la capa privada de red.
- **Sistema de Archivos:** Se configuró Amazon EFS (NFSv4). Este sistema permite que todos los servidores web monten la carpeta /var/www/html/moodle/moodledata como si fuera un disco local, garantizando que los archivos subidos por un alumno sean visibles inmediatamente para todos los nodos del clúster.

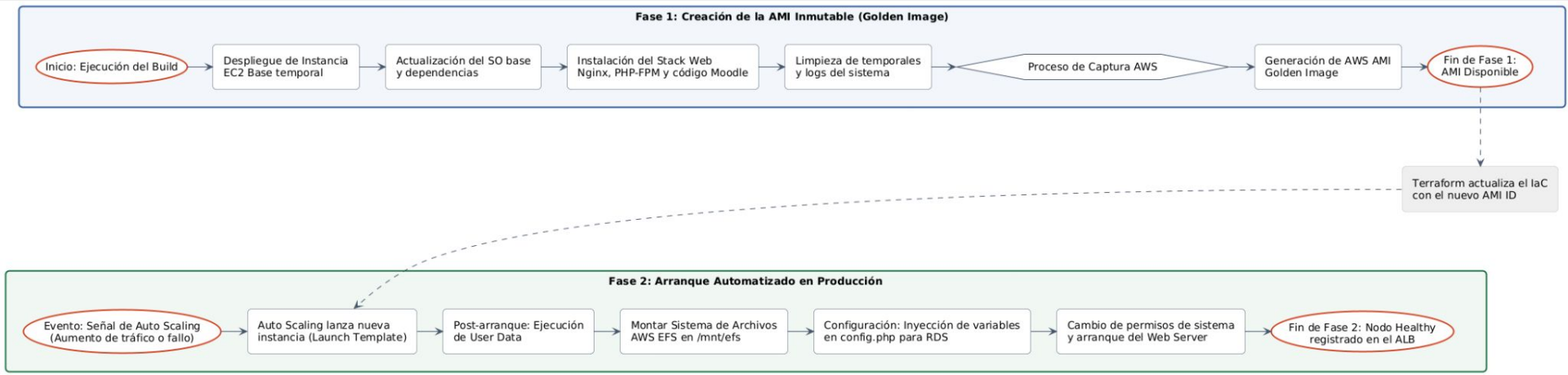
05 DESARROLLO E IMPLEMENTACIÓN TÉCNICA

5.6. Capa de Computación y Automatización

Se realizó una migración conceptual de "servidores mascota" (mantenimiento manual) a "servidores ganado" (efímeros y automáticos).

- **Ingeniería de la Imagen (Golden AMI):** En lugar de configurar cada servidor desde cero, se creó una imagen base a partir de una instancia piloto configurada manualmente, donde se optimizaron los parámetros de PHP y Apache.
- **Plantilla de Lanzamiento (Launch Template):** Se definió la configuración de hardware (t3.micro) y los permisos de seguridad (Roles IAM) necesarios para que las instancias puedan operar sin intervención.
- **Automatización del Arranque (User Data):** Se desarrolló un script en Bash que se inyecta en cada nueva instancia. Este script se encarga de las tareas de "última milla": montar el disco compartido EFS usando el DNS regional y ajustar los permisos de ficheros.

05 DESARROLLO E IMPLEMENTACIÓN TÉCNICA



< **Imagen 11.** Diagrama de flujo mostrando la creación de la AMI y el proceso de arranque automatizado.>

05 DESARROLLO E IMPLEMENTACIÓN TÉCNICA

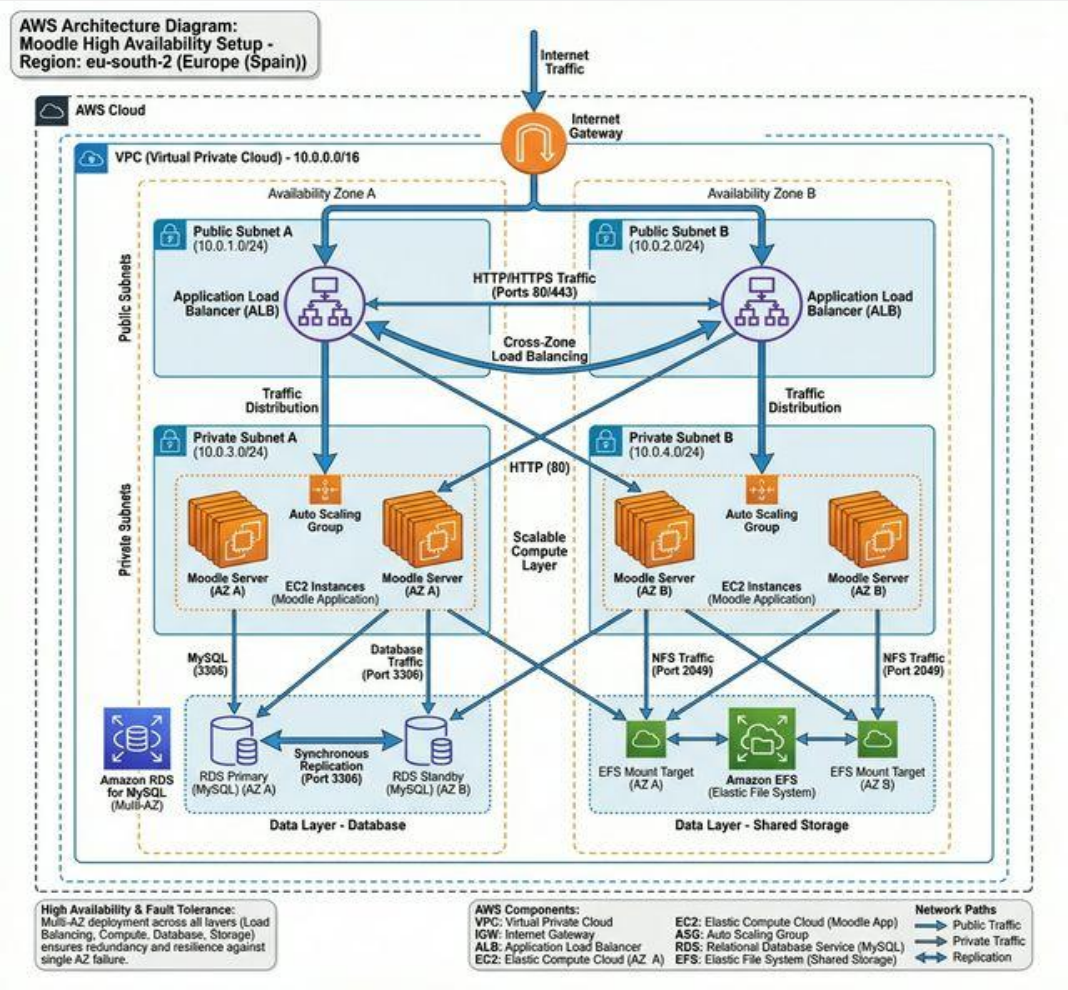
5.7. Orquestación del Sistema: Cómputo, Seguridad Perimetral y Observabilidad

Esta sección describe el núcleo operativo de la plataforma, donde convergen la capacidad de procesamiento, la protección de datos y el monitoreo inteligente. La arquitectura no se comporta como un ente estático, sino como un ecosistema dinámico basado en tres pilares:

- **Capa de Cómputo y Elasticidad:** Se implementó un Auto Scaling Group (ASG) que gestiona instancias EC2 (t3.micro) distribuidas en subredes privadas. La automatización se garantiza mediante un script de User Data que, al nacer cada instancia, realiza la instalación de dependencias y el montaje de los volúmenes EFS de forma autónoma.

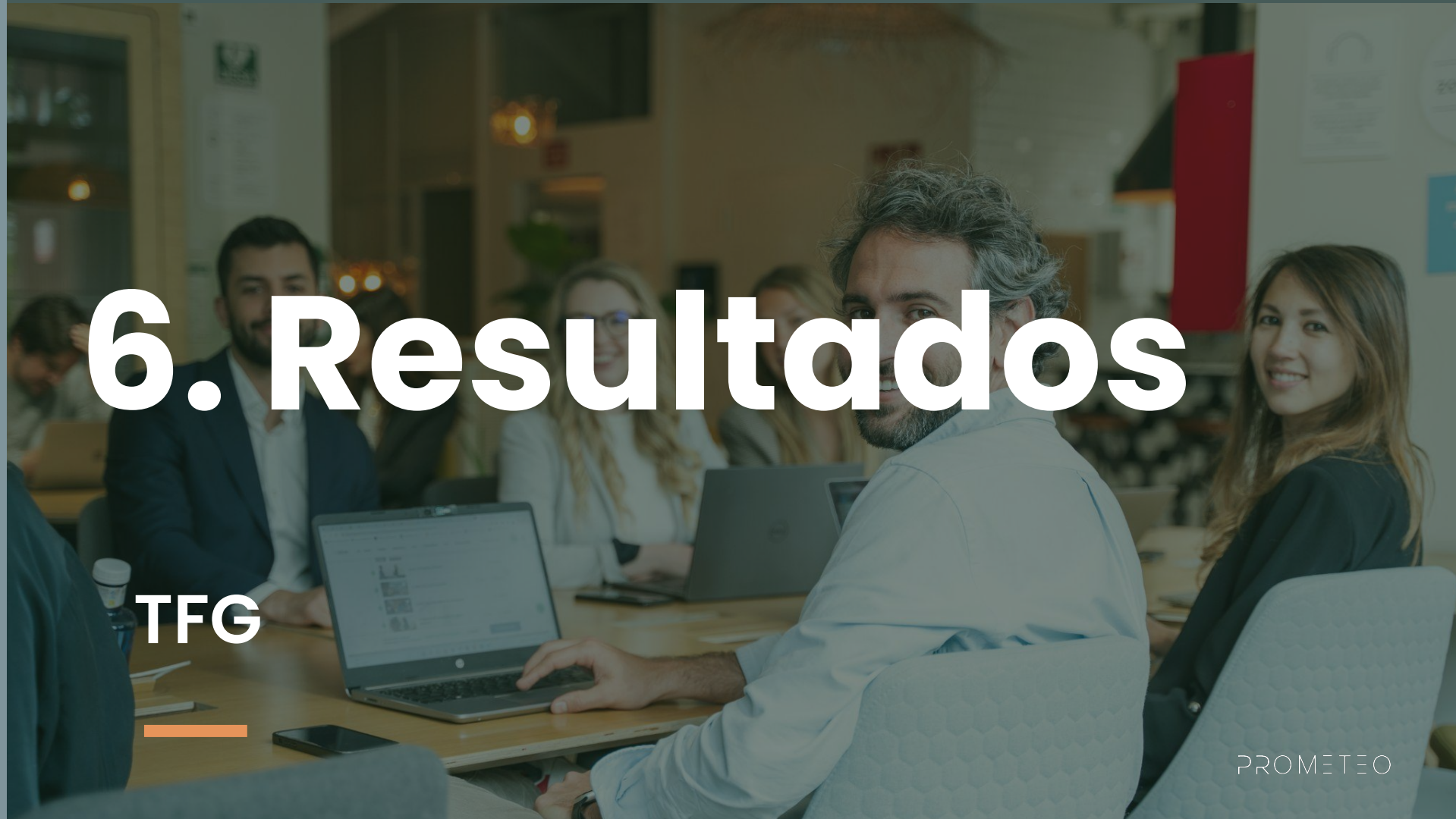
- **Distribución de Tráfico y Seguridad (Cadena de Confianza):** El Application Load Balancer (ALB) actúa como el único punto de entrada (Puerto 80), protegiendo la identidad de los servidores. Se aplicó una política de "Seguridad en Capas" mediante Security Groups: los servidores web solo aceptan tráfico del balanceador, y la base de datos solo acepta peticiones de los servidores web, eliminando cualquier vector de ataque directo desde el exterior.

- **Observabilidad y Auto-recuperación (Self-Healing):** Mediante Amazon CloudWatch, el sistema monitoriza constantemente el estado de salud y la carga de CPU. Si un nodo falla o la demanda aumenta, las alarmas disparan automáticamente la creación de nuevas instancias, asegurando que la plataforma sea resiliente y capaz de recuperarse sin intervención humana.



<Imagen 12: Arquitectura de Referencia: Plataforma Moodle en Alta Disponibilidad y Escalabilidad Horizontal.>

PROMETEO



6. Resultados

TFG

PROMETEO

06 Resultados y validación

6. Capa de Computación y Automatización

Para certificar la robustez del diseño, se sometió la infraestructura a pruebas de estrés ("Chaos Testing"):

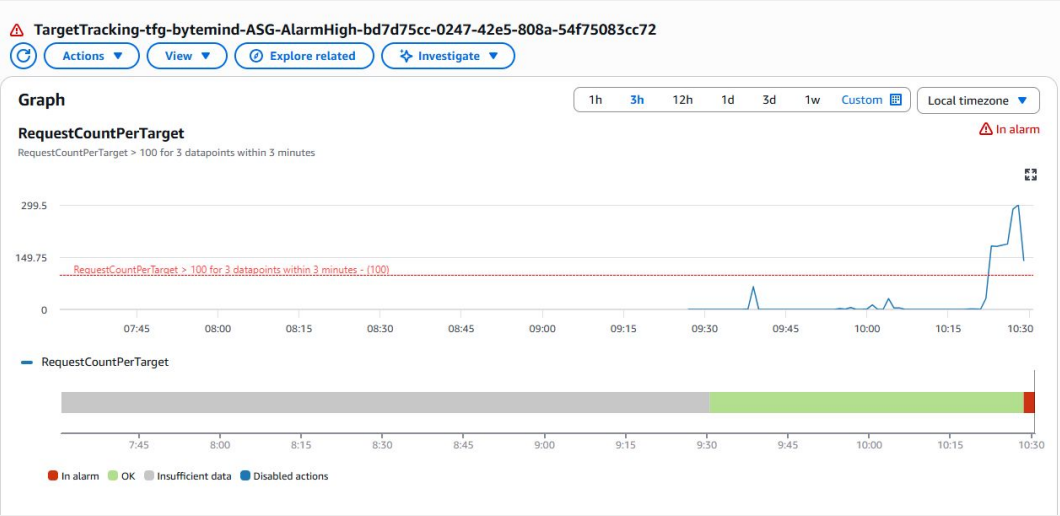
6.1. Prueba de Resiliencia (Self-Healing)

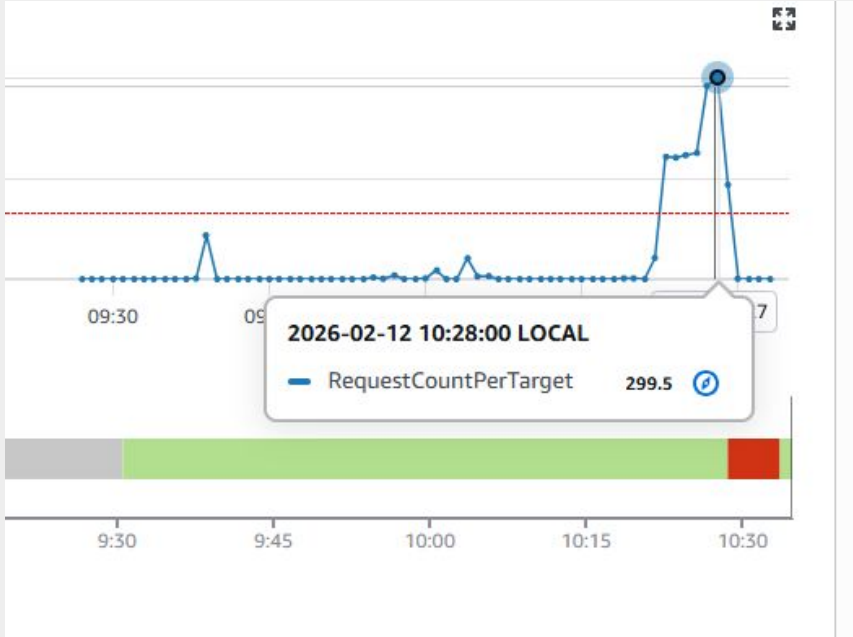
Se forzó la terminación manual de una instancia EC2 activa desde la consola de AWS simulando un fallo de hardware crítico.

- **Comportamiento:** El Balanceador detectó el fallo de salud y cesó el envío de tráfico a la instancia afectada. Simultáneamente, el Grupo de Autoescalado detectó la pérdida de capacidad y aprovisionó una nueva instancia automáticamente.
- **Resultado:** El servicio se mantuvo operativo gracias a la redundancia de zona, y la capacidad total del clúster se recuperó en menos de 3 minutos.

<Imagen 11: Diagrama de topología de red y flujo de datos implementado en AWS.>

06 Resultados y validación





06 Resultados y validación

History (4)

30 days up to current time

Search

1

Date (Local)	Type	Description
2026-02-12 10:28:40	Action	Successfully executed action arn:aws:autoscaling:eu-south-2:495599741055:scalingPolicy:b96564eb-cb43-498b-b42c-ed5bfcb776fd:autoScalingGroupName/tfg-bytemind-ASG:policyName/tfg-bytemind-request-scaling-policy
2026-02-12 10:28:40	State update	Alarm updated from OK to In alarm.
2026-02-12 09:30:40	State update	Alarm updated from Insufficient data to OK.
2026-02-12 09:27:32	Configuration update	Alarm "TargetTracking-tfg-bytemind-ASG-AlarmHigh-bd7d75cc-0247-42e5-808a-54f75083cc72" created

Activar Windows
Ve a Configuración para activar Windows.

Details | Tags | Actions | History | Parent alarms

Actions Actions enabled

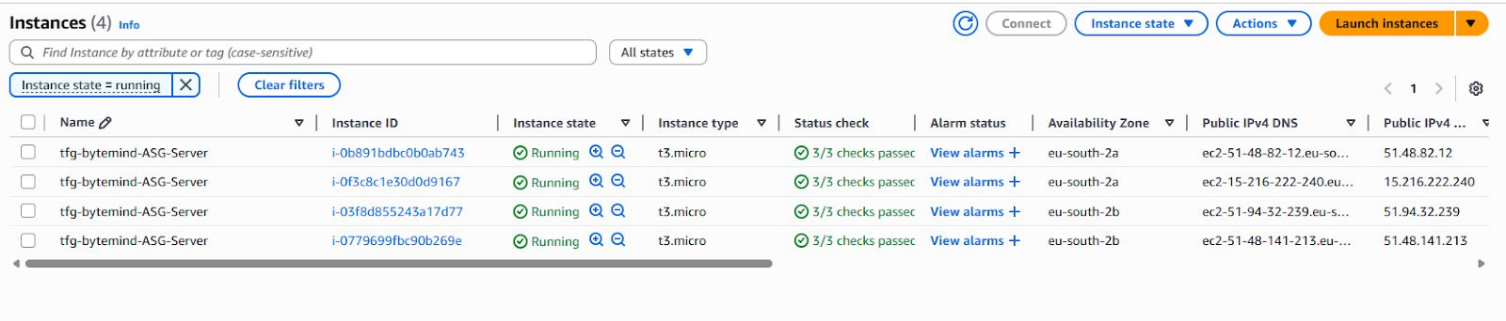
Type	Description
Auto Scaling	When alarm transitions to in alarm, use policy tfg-bytemind-request-scaling-policy (Maintain the metric ALBRequestCountPerTarget at the target value of 100.) for the gr

Activar Windows
Ve a Configuración para activar Windows.

06 Resultados y validación

https://github.com/Madaru69/TFG/blob/main/docs/TESTING_GUIDE.md

06 Resultados y validación



Instances (4) Info

Connect | Instance state | Actions | Launch instances

Find Instance by attribute or tag (case-sensitive) | All states

Instance state = running | Clear filters

1

Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4 DNS	Public IPv4 ...
tfg-bytemind-ASG-Server	i-0b891bdbc0b0ab743	Running	t3.micro	3/3 checks passed	View alarms +	eu-south-2a	ec2-51-48-82-12.eu-so...	51.48.82.12
tfg-bytemind-ASG-Server	i-0f3c8c1e30d0d9167	Running	t3.micro	3/3 checks passed	View alarms +	eu-south-2a	ec2-15-216-222-240.eu...	15.216.222.240
tfg-bytemind-ASG-Server	i-03f8d855243a17d77	Running	t3.micro	3/3 checks passed	View alarms +	eu-south-2b	ec2-51-94-32-239.eu-s...	51.94.32.239
tfg-bytemind-ASG-Server	i-0779699fbc90b269e	Running	t3.micro	3/3 checks passed	View alarms +	eu-south-2b	ec2-51-48-141-213.eu-...	51.48.141.213

https://github.com/Madaru69/TFG/blob/main/docs/TESTING_GUIDE.md

06 Resultados y validación

6.2. Validación de Persistencia

Se verificó que, tras la destrucción y recreación de las instancias de cómputo, los datos de los cursos y los archivos subidos permanecían intactos y accesibles, confirmando el correcto funcionamiento del desacoplamiento en RDS y EFS.

6.3. Rendimiento en la Región España

Las pruebas de acceso desde ubicaciones locales mostraron una reducción significativa de la latencia en comparación con despliegues en regiones de EE. UU., validando la decisión estratégica de activar la región eu-south-2.

06 Resultados y validación

6.4. Estimación de Costes de la Arquitectura Final (FinOps)

- **Inversión Mensual:** 72.31 USD (Arquitectura base en Alta Disponibilidad).
- **Eficiencia de Recursos:** Uso de instancias t3.micro bajo demanda y almacenamiento gp3 de última generación (mejor rendimiento, menor coste).
- **Ahorro en Licenciamiento:** Adopción de MySQL (Open Source), eliminando costes de software propietario.
- **Escalabilidad Inteligente:** El presupuesto es elástico; el coste solo aumenta proporcionalmente ante picos reales de demanda de alumnos.

Export Date: 04/03/2026 **Language:** English

Estimate url

Estimate summary

Upfront cost	Monthly cost	Total 12 months cost
0.00 USD	72.31 USD	867.72 USD Includes upfront cost

Detailed Estimate

Name	Group	Region	Upfront cost	Monthly cost
Amazon EC2	-	Europe (Spain)	0.00 USD	16.64 USD
Status	-			
Description:	-			
Config summary	Tenancy (Shared Instances), Operating system (Linux), Workload (Consistent, Number of instances: 2), Advance EC2 instance (t3.micro), Pricing strategy (On-Demand Utilization: 100 %Utilized/Month), Enable monitoring (disabled)			

Name	Group	Region	Upfront cost	Monthly cost
Elastic Load Balancing	-	Europe (Spain)	0.00 USD	18.63 USD
Status	-			
Description:	-			
Config summary	Number of Application Load Balancers (1)			

Name	Group	Region	Upfront cost	Monthly cost
Amazon Elastic File System (EFS)	-	Europe (Spain)	0.00 USD	1.65 USD
Status	-			
Description:	-			
Config summary	Desired Storage Capacity (5 GB per month)			

Name	Group	Region	Upfront cost	Monthly cost
Amazon RDS for MySQL	-	Europe (Spain)	0.00 USD	35.39 USD
Status	-			
Description:	-			
Config summary	Storage amount (20 GB), Storage for each RDS instance (General Purpose SSD (gp3)), Nodes (1), Instance type (db.t3.micro), Utilization (On-Demand only) (100 %Utilized/Month), Deployment option (Single-AZ), Pricing strategy (OnDemand)			

PROMETEO

7. CONCLUSIONES

TFG

PROMETEO

07 Conclusiones

7. Conclusiones del proyecto:

La transición de un modelo monolítico a una arquitectura Cloud-Native ha demostrado el cumplimiento íntegro de los objetivos planteados:

- **Resiliencia Empírica (0 SPOF):** Las pruebas de caos demostraron que el diseño Multi-AZ y el Self-healing recuperan el servicio en menos de 3 minutos ante caídas críticas, garantizando la continuidad educativa.
- **Seguridad por Diseño:** El aislamiento en subredes privadas, la "Cadena de Confianza" de los Security Groups y el uso de IAM Roles han neutralizado vectores de ataque directos sin comprometer la conectividad.
- **Eficiencia FinOps:** La auditoría de la infraestructura permitió eliminar el 100% de las IPs públicas en la capa de cómputo, optimizando el rendimiento financiero sin perder escalabilidad.
- **Gobernanza mediante IaC:** Terraform transformó un despliegue complejo y propenso a fallos manuales en un proceso determinista, auditable y capaz de aprovisionarse en minutos.

8. LÍNEAS FUTURAS

TFG

PROMETEO

08 Líneas de Trabajo Futuro

8. Líneas de Trabajo Futuro:

Para elevar la plataforma de un entorno altamente disponible a una solución de grado de producción corporativo, se plantean los siguientes hitos:

- **Cifrado Web y Dominio Personalizado:** Transición de tráfico HTTP a tráfico seguro (HTTPS) mediante AWS Certificate Manager (ACM), y configuración de un nombre de dominio corporativo utilizando Amazon Route 53.
- **Gestión de Configuración (Infraestructura Inmutable):** Transición de los scripts de arranque (Bash/User Data) hacia Ansible y Packer para la creación de imágenes (AMIs) preconfiguradas y optimizadas.
- **Integración y Despliegue Continuo (CI/CD):** Evolución de la ejecución de Terraform vía CLI local hacia pipelines automatizados (ej. GitHub Actions o AWS CodePipeline) para validación de código.
- **Identidad Corporativa:** Integración del sistema base de Moodle con servicios de directorio externo (LDAP / Active Directory) para habilitar inicio de sesión único (SSO).

9. Bibliografía

TFG

PROMETEO

09 Bibliografía

9. Bibliografía:

Normativa y Estándares:

- Agencia Española de Protección de Datos (AEPD). (2023). *Guía para clientes que contraten servicios de Cloud Computing*. Madrid: AEPD. https://www.oas.org/es/sla/ddi/docs/Gu%C3%ADa%20para%20clientes%20que%20contraten%20servicios%20de%20Cloud%20Computing%20-%20AGPD.pdf
- Esquema Nacional de Seguridad (ENS). (2022). *Real Decreto 311/2022, de 3 de mayo, por el que se regula el Esquema Nacional de Seguridad*. Boletín Oficial del Estado. https://www.boe.es/buscar/act.php?id=BOE-A-2022-7191

Metodologías y Conceptos Arquitectónicos:

- Amazon Web Services. (2024). *AWS Well-Architected Framework*. Recuperado de https://docs.aws.amazon.com/wellarchitected/latest/framework/welcome.html
- FinOps Foundation. (2024). *What is FinOps? Cloud Financial Management*. Recuperado de https://www.finops.org/introduction/what-is-finops/
- Principles of Chaos Engineering. (n.d.). *Principles of Chaos Engineering*. Recuperado de https://principlesofchaos.org

09 Bibliografía

9. Bibliografía:

Documentación Técnica de Infraestructura (AWS & HashiCorp):

- Amazon Web Services. (2025). *Security best practices in IAM*. Recuperado de https://docs.aws.amazon.com/IAM/latest/UserGuide/best-practices.html
- Amazon Web Services. (2025). *Amazon VPC User Guide (Security Groups & Subnetting)*. Recuperado de https://docs.aws.amazon.com/vpc/latest/userguide/
- Amazon Web Services. (2025). *High availability (Multi-AZ) for Amazon RDS*. Recuperado de https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Concepts.MultiAZ.html
- Amazon Web Services. (2025). *Amazon Elastic File System (EFS) User Guide*. Recuperado de https://docs.aws.amazon.com/efs/
- Amazon Web Services. (2025). *Amazon EC2 Auto Scaling User Guide*. Recuperado de https://docs.aws.amazon.com/autoscaling/ec2/userguide/
- Amazon Web Services. (2025). *Amazon CloudWatch User Guide*. Recuperado de https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/
- HashiCorp. (2025). *Terraform: AWS Provider Documentation*. Recuperado de https://registry.terraform.io/providers/hashicorp/aws/latest/docs

09 Bibliografía

9. Bibliografía:

Recursos de Aplicación, Rendimiento y Análisis de Costes:

- Amazon Web Services. (2026). *AWS Pricing Calculator*. Recuperado de https://calculator.aws/
- Moodle HQ. (2024). *Performance recommendations*. Recuperado de https://docs.moodle.org/en/Performance_recommendations
- Moodle HQ. (2022). *User site capacities*. Recuperado de https://docs.moodle.org/en/User_site_capacities
- Bitnami. (2023). *Bitnami Moodle Stack Architecture Documentation*. Recuperado de https://docs.bitnami.com/
- Vantage. (2025). *Amazon EC2 t3.micro Pricing*. Recuperado de https://instances.vantage.sh/aws/ec2/t3.micro?currency=USD

9. ANEXOS

TFG

PROMETEO

09 ANEXOS

9.1 Anexos

- Anexo I: Glosario de Términos Cloud y DevOps
- Anexo II: Prototipo Manual Fase 1 (Monolito con Bitnami)
- Anexo III: Estructura del Proyecto de Infraestructura como Código (Terraform)
- Anexo IV: Script de Automatización de Arranque (User Data)
- Anexo V: Políticas de Seguridad y Roles
- Anexo VI: Estimación Financiera y FinOps
- Anexo VII: Evidencias de Resiliencia y Pruebas de Caos

PROMETEO